

```

to process
  repeat NCandidates - 2 [
    apply_nextPref
    compute
  ]
end

```

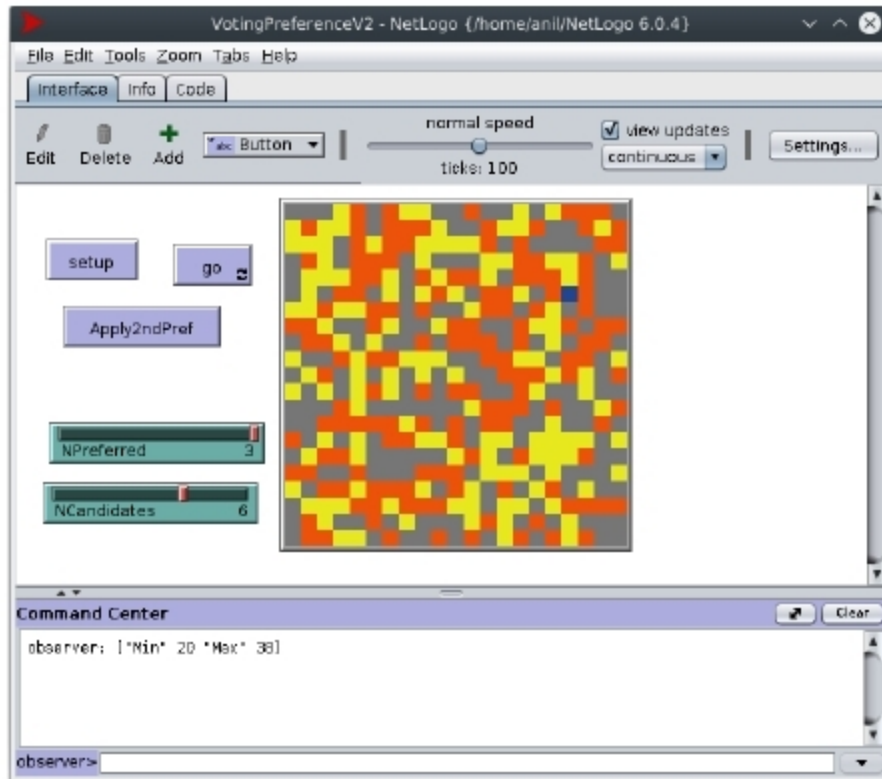


Figure 2: The results after the main vote. The result is for six candidates, where three of them have double the chance of winning. The screen is dominated by three colours.

The hard part is handling the votes for candidates who have been eliminated. In the first preference votes, the item in the list itself is removed. In the second preference votes, it is simpler to mark eliminated candidates' votes by the place holder -1.

```

to apply_nextPref
  ask patches [
    ; select the losing candidate
    set loser position min votes votes
    ; remove his votes
    set votes remove-item loser votes
    ; remove 2nd pref votes
    remove-loser-nextPref
    set listc remove-item loser listc
    ; -1 votes is used for eliminated candidates
    let votes2 filter [ i -> i != -1 ] item loser nextPref
    ; transfer 2nd pref votes to the remaining candidates
    set votes (map + votes votes2)
  ]
end

to remove-loser-nextPref
  ; replace votes of losing candidate by -1 in each list
  let c item loser listc
  let tempList []

```

```

foreach nextPref [ x ->
  let v2 replace-item c x -1
  set tempList lput v2 tempList
]
set nextPref tempList
end

```

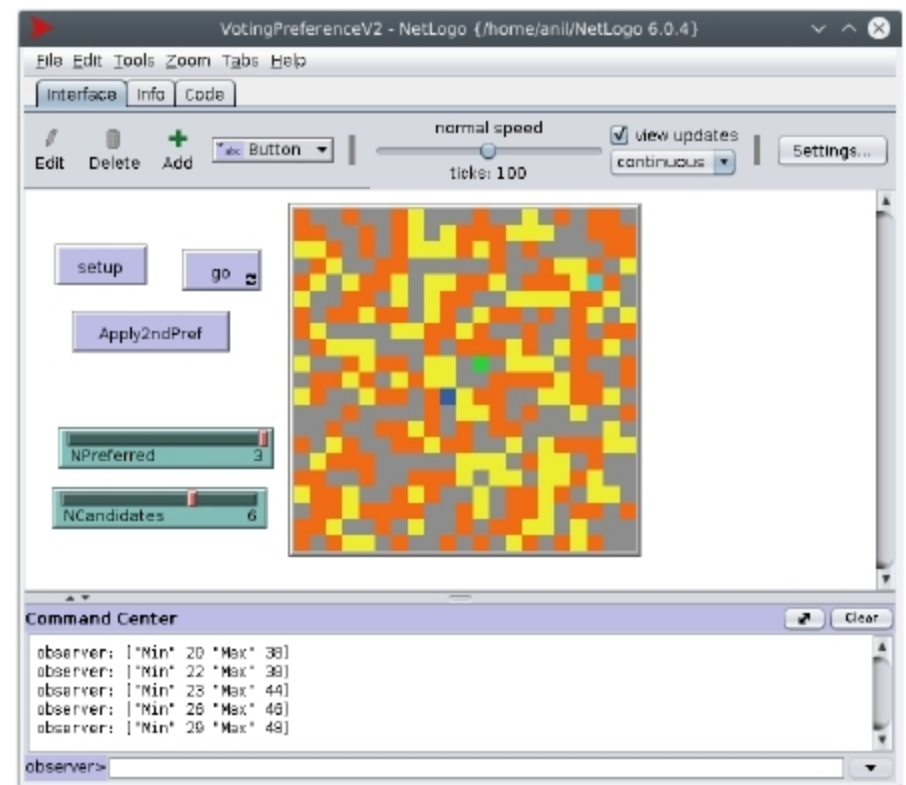


Figure 3: The result after the second preference vote is counted for the same run as in Figure 2. The screen is still dominated by three colours but there is a substantial difference from Figure 2.

Figure 3 shows the results after the second preference votes are counted. This is not a serious model, but just to check and demonstrate that the results can be very different if second preference votes are applied.

After the second preference votes are applied, the winners will be closer to crossing the 50 per cent mark, though not always, as some second preference votes would have been for the candidates who are eliminated.

The model chosen was very simple so that the ideas of how to model in NetLogo can be explored. Greater complexity can be added, and it is fun to program in a language with syntax different from the procedural languages.

One aspect of NetLogo that we have not used or explored is the use of turtles. Each candidate could be a turtle, belonging to a party or could be an independent. The model can be mind-bogglingly complex. You can go ahead and create one in which your favourite party wins! **END** 🐧

By: Dr Anil Seth

The author has earned the right to do what interests him. You can find him online at <http://sethanil.com> and <http://sethanil.blogspot.com>, or you can reach him via email at anil@sethanil.com.